ЛАБОРАТОРНА РОБОТА №2

СИСТЕМНЕ ПРОГРАМУВАННЯ

ПАРАЛЕЛІЗМ ДАНИХ ТА ПАРАЛЕЛІЗМ ЗАДАЧ (Ч.1)

Заяць Діани Юріївни

Хід виконання роботи:

У програмі реалізовано консольне меню, яке дозволяє запускати окремо кожне із чотирьох завдань. Головна логіка міститься у файлі Program.cs. У методі Main() використовується нескінченний цикл while(true), завдяки якому після завершення будь-якого завдання користувач повертається до меню. Вибір пункту меню зчитується за допомогою Console.ReadLine(), після чого через оператор switch викликається відповідний метод Run() для конкретного завдання.

Після завершення виконання вибраного пункту програма виводить повідомлення про завершення роботи, очікує натискання клавіші користувачем та очищає консоль для повторного показу меню.

```
0 references
private static void Main(string[] args)
{
    // Головний цикл меню: працює, доки користувач не вибере вихід.
    while (true)
    {
        // Перед кожним введенням показуємо доступні пункти.
        Console.WriteLine("1 - Task 1");
        Console.WriteLine("2 - Task 2");
        Console.WriteLine("3 - Task 3");
        Console.WriteLine("4 - Task 4");
        Console.WriteLine("0 - Exit");
        Console.Write("Choose an option: ");

        string choice = Console.ReadLine();
        if (choice == "0")
        {
            // Єдина умова повного завершення програми.
            return;
        }

        // Запускаємо потрібне завдання за вибором користувача.
        switch (choice)
        {
            case "1":
                Task1Demo.Run();
                break;
            case "2":
                Task2Demo.Run();
                break;
            case "3":
                Task3Demo.Run();
                break;
            case "4":
                Task4Demo.Run();
                break;
            default:
                Console.WriteLine("Invalid choice. Try again.");
                break;
        }

        // Пауза після виконання, щоб користувач встиг побачити результат.
        Console.WriteLine("Task completed.");
        Console.WriteLine("Press any key to return to the menu...");
        Console.ReadKey();
        Console.Clear();
    }
```

Рис. 1.1 – Код головного меню програми

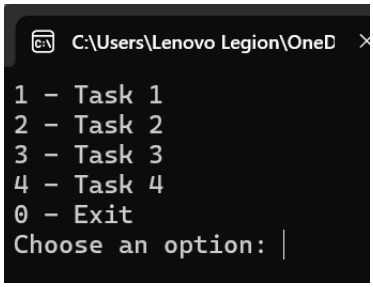

Рис. 1.2 – Головне меню програми під час виконання

2.1. Створення та запуск паралельних задач

Для виконання першого завдання було створено клас Task1Demo. У методі Run() спочатку виводиться заголовок завдання, після чого створюються дві окремі задачі:

- task1 виконує метод PrintNumbers();
- task2 виконує метод PrintLetters().

Обидві задачі створюються через конструктор класу Task, але не запускаються автоматично. Для початку їх виконання використовується метод Start(). Таким чином, після виклику task1.Start() та task2.Start() обидві задачі починають працювати паралельно.

Метод PrintNumbers() містить цикл від 1 до 10. На кожній ітерації в консоль виводиться поточне число, після чого виконується затримка Thread.Sleep(200). Метод PrintLetters() аналогічно містить цикл по символах від A до J, виводить кожну літеру та робить паузу 200 мс.

Після запуску задач у методі Run() використовується Task.WaitAll(task1, task2). Цей виклик призупиняє подальше виконання поточного потоку до того моменту, поки обидві задачі не завершаться. Після цього виводиться повідомлення про завершення роботи двох задач.

Паралельність тут проявляється у тому, що рядки з числами та літерами можуть з’являтися в консолі у змішаному порядку. Це означає, що задачі виконуються одночасно, а не послідовно.

```
1 reference
internal static class Task1Demo
{
    1 reference
    public static void Run()
    {
        Console.WriteLine("--- Task 1 ---");

        // Створюємо дві окремі задачі через конструктор Task.
        // Тут НЕ використовуємо Task.Run, щоб виконати вимогу лабораторної.
        Task task1 = new Task(PrintNumbers);
        Task task2 = new Task(PrintLetters);

        // Явно запускаємо обидві задачі методом Start().
        task1.Start();
        task2.Start();

        // Очікуємо завершення обох задач.
        // Без WaitAll метод міг би завершитися раніше за виконання задач.
        Task.WaitAll(task1, task2);
        Console.WriteLine("Both tasks have completed.");
    }

    1 reference
    private static void PrintNumbers()
    {
        // Вивід чисел від 1 до 10 із паузою 200 мс.
        for (int i = 1; i <= 10; i++)
        {
            Console.WriteLine("Numbers Task: " + i);
            Thread.Sleep(200);
        }
    }

    1 reference
    private static void PrintLetters()
    {
        // Вивід літер від A до J із паузою 200 мс.
        for (char letter = 'A'; letter <= 'J'; letter++)
        {
            Console.WriteLine("Letters Task: " + letter);
            Thread.Sleep(200);
        }
    }
```

Рис. 2.1.1 – Код створення та запуску двох паралельних задач

```
2 - Task 2
3 - Task 3
4 - Task 4
0 - Exit
Choose an option: 1
--- Task 1 ---
Letters Task: A
Numbers Task: 1
Letters Task: B
Numbers Task: 2
Letters Task: C
Numbers Task: 3
Letters Task: D
Numbers Task: 4
Letters Task: E
Numbers Task: 5
Letters Task: F
Numbers Task: 6
Letters Task: G
Numbers Task: 7
Letters Task: H
Numbers Task: 8
Numbers Task: 9
Letters Task: I
Letters Task: J
Numbers Task: 10
Both tasks have completed.
Task completed.
Press any key to return to the menu..
```

Рис. 2.1.2 – Результат виконання

Поясніть різницю між класами Thread та Task.

Клас Thread представляє окремий системний потік виконання. Його використання дає низькорівневий контроль над потоком, але потребує більшої кількості ручного керування. Клас Task є вищорівневою абстракцією, яка представляє асинхронну або паралельну операцію. Task не обов'язково відповідає окремому потоку, оскільки планування виконання здійснюється через внутрішні механізми .NET, зокрема через пул потоків. Основна перевага Task полягає у спрощенні паралельного програмування: задачі легше запускати, очікувати, комбінувати, будувати продовження та отримувати результати. Тому в більшості випадків Task є зручнішим і сучаснішим засобом, ніж Thread.

2.2. Робота з ідентифікаторами задач

Для виконання другого завдання було створено клас Task2Demo. У методі Run() створюються три окремі задачі, кожна з яких викликає один і той самий метод CountToFive(). Перед запуском у консоль виводяться значення властивості Id для кожної створеної задачі.

Після цього всі три задачі запускаються методом Start(), а завершення їх виконання очікується за допомогою Task.WaitAll(task1, task2, task3).

Метод CountToFive() реалізує цикл від 1 до 5. На кожному кроці в консоль виводиться номер поточного кроку та значення Task.CurrentId, що дозволяє побачити, яка саме задача виконує код у даний момент часу.

Властивість Id належить конкретному об'єкту Task і задається в момент створення задачі. Вона використовується для унікального розпізнавання задачі. Властивість Task.CurrentId показує ідентифікатор тієї задачі, яка виконується саме зараз. Тому її доцільно використовувати всередині методу, що виконується паралельно.

Це дозволяє простежити, яка задача виконує певну дію, а також побачити, як саме розподіляється виконання між паралельними задачами.

```
namespace lab2
{
    1 reference
    internal static class Task2Demo
    {
        1 reference
        public static void Run()
        {
            Console.WriteLine("--- Task 2 ---");

            // Створюємо три окремі задачі з однаковим тілом виконання.
            Task task1 = new Task(CountToFive);
            Task task2 = new Task(CountToFive);
            Task task3 = new Task(CountToFive);

            // Показуємо системно згенеровані Id кожної створеної задачі.
            // Ці значення присвоюються рантаймом і можуть відрізнятися між запусками.
            Console.WriteLine("Task 1 Id = " + task1.Id);
            Console.WriteLine("Task 2 Id = " + task2.Id);
            Console.WriteLine("Task 3 Id = " + task3.Id);

            // Запускаємо всі задачі паралельно.
            task1.Start();
            task2.Start();
            task3.Start();

            // Чекаємо завершення всіх трьох задач.
            Task.WaitAll(task1, task2, task3);
        }

        3 references
        private static void CountToFive()
        {
            // Усередині задачі показуємо Task.CurrentId та поточний крок.
            // Це дозволяє бачити, яка саме задача виконує конкретну ітерацію.
            for (int step = 1; step <= 5; step++)
            {
                Console.WriteLine("Running task CurrentId = " + Task.CurrentId + ", step = " + step);
                Thread.Sleep(200);
            }
        }
    }
}
```

Рис. 2.2.1 – Код ідентифікатора задач

```
--- Task 2 ---
Task 1 Id = 3
Task 2 Id = 4
Task 3 Id = 5
Running task CurrentId = 3, step = 1
Running task CurrentId = 4, step = 1
Running task CurrentId = 5, step = 1
Running task CurrentId = 4, step = 2
Running task CurrentId = 5, step = 2
Running task CurrentId = 3, step = 2
Running task CurrentId = 5, step = 3
Running task CurrentId = 4, step = 3
Running task CurrentId = 3, step = 3
Running task CurrentId = 3, step = 4
Running task CurrentId = 5, step = 4
Running task CurrentId = 4, step = 4
Running task CurrentId = 4, step = 5
Running task CurrentId = 5, step = 5
Running task CurrentId = 3, step = 5
Task completed.
Press any key to return to the menu...
```

Рис. 2.2.2 –Результат виконання

Чи є ідентифікатори задач впорядкованими? Як їх можна використати для відстеження виконання?

Ідентифікатори задач є унікальними, але не гарантується, що вони будуть впорядкованими або послідовними. Значення Id призначається кожній задачі під час її створення, однак порядок виконання задач не залежить від значення ідентифікатора. Ідентифікатори задач можна використовувати для відстеження паралельного виконання програми. Наприклад, за допомогою Id можна визначити, які задачі були створені, а за допомогою Task.CurrentId – з'ясувати, яка задача виконує конкретний фрагмент коду в даний момент. Це корисно під час налагодження, логування та аналізу поведінки багатозадачної програми.

2.3. Використання ContinueWith() для продовження задачі

Для виконання третього завдання було створено клас Task3Demo. У методі Run() користувач вводить значення N з клавіатури. Після цього виконується перевірка правильності введення за допомогою int.TryParse(...). Якщо введене значення некоректне або не є додатним, програма виводить повідомлення про помилку та завершує виконання поточного завдання. Далі створюється задача типу Task<int>, яка викликає метод CalculateSum(n). Цей метод виконує обчислення суми чисел від 1 до N та повертає результат типу int. Після створення основної задачі формується продовження через ContinueWith(...). Як продовження використовується лямбда-вираз, який отримує попередню задачу як параметр, звертається до її властивості Result і виводить обчислену суму в консоль.

Потім основна задача запускається методом Start(), а завершення як обчислення, так і продовження очікується за допомогою Task.WaitAll(sumTask, continuationTask).

Особливість цього завдання полягає в тому, що тут використовується задача з результатом – Task<int>. Це дозволяє не просто виконати обчислення паралельно, а й отримати результат роботи задачі.

Метод ContinueWith() дозволяє явно задати наступний етап виконання після завершення попередньої задачі. Таким чином логіка програми стає послідовною за змістом:

1. спочатку обчислити суму;
2. потім вивести результат.

Використання лямбда-виразу робить код коротшим і зручнішим, оскільки немає потреби створювати окремий метод для продовження.

```
1 reference
public static void Run()
{
    Console.WriteLine("--- Task 3 ---");
    Console.WriteLine("Enter N:");

    string input = Console.ReadLine();
    // Перевіряємо, що введено додатне ціле число.
    if (!int.TryParse(input, out int n) || n <= 0)
    {
        Console.WriteLine("Invalid value.");
        return;
    }

    Console.WriteLine("Calculating the sum from 1 to N...");

    // Основна задача обчислює суму і повертає int-результат.
    Task<int> sumTask = new Task<int>(() => CalculateSum(n));
    // Продовження (лямбда) виконується після sumTask і друкує результат.
    // previousTask — це посилання на завершену попередню задачу.
    Task continuationTask = sumTask.ContinueWith(previousTask =>
    {
        Console.WriteLine("Result: " + previousTask.Result);
    });

    // Запускаємо головну задачу; continuation стартує автоматично після неї.
    sumTask.Start();

    // Очікуємо і обчислення, і continuation.
    Task.WaitAll(sumTask, continuationTask);
}

1 reference
private static int CalculateSum(int n)
{
    // Явне обчислення суми циклом від 1 до N.
    // Такий підхід прозоріший для навчальної демонстрації.
    int sum = 0;
    for (int i = 1; i <= n; i++)
    {
        sum += i;
    }

    return sum;
}
```

Рис. 2.3.1 - Код створення і продовження задачі з обчисленням

```
1 - Task 1
2 - Task 2
3 - Task 3
4 - Task 4
0 - Exit
Choose an option: 3
--- Task 3 ---
Enter N:
32
Calculating the sum from 1 to N...
Result: 528
Task completed.
Press any key to return to the menu...
```

Рис. 2.3.2 – Результат виконання

Які переваги має використання продовжень задач у порівнянні з класичним підходом синхронізації?

Продовження задач дозволяють природно описати залежність між етапами виконання програми: одна задача автоматично запускається після завершення іншої. Це зменшує потребу в ручній синхронізації через блокування, сигнали, події або додаткові перевірки стану.

Основні переваги ContinueWith():

- спрощення структури коду;
- явне задання послідовності дій;
- зменшення кількості службового коду для синхронізації;
- краща читабельність і підтримуваність програми;
- зручне поєднання асинхронних і паралельних етапів обробки.

Таким чином, продовження задач роблять програму більш зрозумілою, компактною та менш схильною до помилок синхронізації.

2.4. Паралельне виконання методів через Parallel.Invoke()

Для виконання четвертого завдання було створено клас Task4Demo. У методі Run() користувач вводить два значення:

- число для обчислення факторіала;
- число N для обчислення суми.

Обидва значення перевіряються на коректність за допомогою int.TryParse(...). Якщо введення неправильне, програма виводить повідомлення про помилку та припиняє виконання цього пункту.

Після цього використовується метод Parallel.Invoke(...), якому передаються три делегати:

- перший викликає метод CalculateFactorial(factorialNumber);
- другий викликає метод CalculateSum(sumNumber);
- третій викликає метод PrintMessages().

Метод CalculateFactorial(int number) обчислює факторіал числа за допомогою циклу та виводить результат у консоль. Метод CalculateSum(int n) обчислює суму чисел від 1 до n. Метод PrintMessages() декілька разів виводить службові повідомлення в консоль із затримкою 300 мс, щоб наочно показати паралельність виконання.

Після завершення всіх трьох методів у консоль виводиться повідомлення про завершення роботи.

Метод Parallel.Invoke() призначений для одночасного запуску кількох методів, які не приймають параметрів і не повертають значення безпосередньо в точку виклику. У цьому випадку параметри передаються через лямбда-вирази.

Перевага такого підходу полягає в тому, що немає потреби вручну створювати кілька об’єктів Task, запускати їх та чекати завершення через WaitAll(). Метод Parallel.Invoke() сам організовує паралельний запуск і автоматично очікує завершення всіх переданих методів.

```
public static void Run()
{
    Console.WriteLine("--- Task 4 ---");
    Console.WriteLine("Enter number for factorial:");
    string factorialInput = Console.ReadLine();

    Console.WriteLine("Enter N for sum:");
    string sumInput = Console.ReadLine();

    // Обидва значення мають бути коректними додатними цілими числами.
    if (!int.TryParse(factorialInput, out int factorialNumber) ||
        !int.TryParse(sumInput, out int sumNumber) ||
        factorialNumber <= 0 ||
        sumNumber <= 0)
    {
        Console.WriteLine("Invalid value.");
        return;
    }

    // Паралельно запускаємо три незалежні методи.
    // Invoke повернеться лише тоді, коли завершаться всі три делегати.
    Parallel.Invoke(
        () => CalculateFactorial(factorialNumber),
        () => CalculateSum(sumNumber),
        PrintMessages
    );

    Console.WriteLine("All methods have completed.");
}

1 reference
private static void CalculateFactorial(int number)
{
    // Для факторіала використовуємо long, щоб уникнути дуже раннього переповнення int.
    long result = 1;
    for (int i = 1; i <= number; i++)
    {
        result *= i;
    }

    Console.WriteLine("Factorial(" + number + ") = " + result);
}
```

```
1 reference
private static void CalculateSum(int n)
{
    // Обчислюємо суму від 1 до N звичайним циклом.
    int sum = 0;
    for (int i = 1; i <= n; i++)
    {
        sum += i;
    }

    Console.WriteLine("Sum 1.." + n + " = " + sum);
}

private static void PrintMessages()
{
    // Допоміжний метод, що імітує окрему роботу з паузою.
    // Дає змогу візуально побачити одночасне виконання з іншими методами.
    for (int i = 0; i < 5; i++)
    {
        Console.WriteLine("Message method is running...");
        Thread.Sleep(300);
    }
}
```

Рис. 2.4.1 – Код паралельного виконання методів

```
Choose an option: 4
--- Task 4 ---
Enter number for factorial:
6
Enter N for sum:
12
Factorial(6) = 720
Sum 1..12 = 78
Message method is running...
Message method is running...
Message method is running...
Message method is running...
Message method is running...
All methods have completed.
Task completed.
Press any key to return to the menu...
```

Рис. 2.4.2 – Результат виконання

Які переваги має використання Parallel.Invoke() у порівнянні зі створенням окремих задач?

Parallel.Invoke() зручний у тих випадках, коли потрібно просто виконати кілька незалежних методів паралельно без необхідності вручну керувати кожною задачею окремо.

Основні переваги Parallel.Invoke():

- простіший і коротший код;
- автоматичне очікування завершення всіх переданих методів;
- відсутність необхідності окремо викликати Start() та WaitAll();
- зручність для паралельного запуску невеликої групи незалежних дій;
- автоматичне використання доступних ресурсів процесора.

Отже, Parallel.Invoke() є зручним засобом для швидкої організації паралельного виконання, якщо не потрібне гнучке керування об’єктами Task.

Висновок:

У ході виконання лабораторної роботи було досліджено основні засоби організації паралельного виконання в C# за допомогою бібліотеки TPL. Було

реалізовано створення та запуск задач, очікування завершення кількох задач, використання ідентифікаторів задач, роботу з задачами, що повертають результат, а також механізм продовження задач і паралельний запуск методів через Parallel.Invoke().

У результаті виконання лабораторної роботи було отримано практичні навички використання класу Task, методів Start(), WaitAll(), ContinueWith() та Parallel.Invoke(). Також було встановлено, що бібліотека TPL значно спрощує розробку паралельних програм у порівнянні з низькорівневим керуванням потоками через клас Thread.